

1 Premier programme

```
/* Source code (hello.c) */
#include <stdio.h>
```

```
int main(){
    printf("Hello_World!\n");
    return 0;
}
```

Compilation : dans un *terminal* :

```
clang-14 hello.c -o hello
```

On peut remplacer clang par gcc et ajouter les options -Wall ou -Werror.

Exécution : si la compilation réussit (sans erreur), on peut alors exécuter ce binaire, toujours dans un terminal, avec ./hello.

2 Structure générale d'un programme

(ici, un seul fichier source, les seules inclusions sont des entêtes de bibliothèques standard)

```
#include <stdio.h> // en-têtes de bibliothèques (sans ;)
#define TAILLE 100 // constante symbolique
```

```
int c; // définitions de variables globales (optionnel)
```

```
int mafonction(char u){ // définitions de fonctions (optionnel)
    ...
    return -42;
}
```

```
int main(){ // fonction principale du programme (obligatoire)
    mafonction('u');
    ...
    return 0; // ou EXIT_SUCCESS
}
```

3 Types de données

- **Types de base :** int (entier signé), char (caractère, sur 1 octet), bool (avec stdbool.h)
- **Type composé :** les tableaux statiques : int tab[12] par exemple.

4 Boucles for

Ces boucles sont à utiliser en priorité lorsque l'on connaît à l'avance le nombre d'itérations.

Exemple : somme des éléments de 1 à 12 :

```
int i;
int s = 0;
for (i=1; i<=12; i++) // attention aux ;
    s = s + i;
```

S'il y a plusieurs instructions dans le *corps*, alors les accolades sont utiles :

```
for (int j=12; j>=0; j--) { // boucle décroissante
    // instruction1
    // instruction2
}
```

5 Boucles while

Lorsque la condition est plus complexe, et qu'on ne peut pas facilement borner le nombre d'itérations :

```
int c = 0;
while(b > 1){
    b = b/2;
    c++;
} // calcule le log_2 de b.
```

La condition du while, qui est une *condition booléenne* (s'évalue à vrai/faux) peut être composée avec non (!), ou (|), et (&&). Attention aux parenthèses.

6 Fonctions

Une fonction sert à créer une portion de code que l'on peut appeler plusieurs fois.

Déclaration et implémentation d'une fonction *nommée* mafonction, qui a un unique paramètre de type caractère, et qui renvoie ("retourne") un entier (son *type de retour* est int) :

```
int mafonction(char c){
    return -42;
}
```

Une fonction peut ne rien retourner, le type de retour est alors void. Une fonction sans paramètre (appelée *procédure*) a void tout seul entre les parenthèses. (En cas de parenthèses "sans void", elle accepte n'importe quel nombre de paramètres). Les modifications de paramètres (sauf tableaux) ne sont pas enregistrées à l'extérieur de la fonction.

Appel d'une fonction (à l'intérieur d'une autre fonction, le main, par exemple) :

```
int resu; // variable du type de retour
resu = mafonction('u'); // le paramètre formel est remplacé par une valeur
```

Alternativement, on peut utiliser le résultat directement :

```
printf("Le_resultat_est_=%d", mafonction('Z')); // %d car le résultat est un entier (int)!
```

7 Tableaux

Déclaration d'un tableau :

```
int t[12]; // comme variable locale d'une fonction
```

ou alors, avec une constante symbolique

```
#define N 100 // au début du programme
...
int tab[N]; // quelque part dans une fonction
```

⚠ Le tableau n'est **pas** automatiquement initialisé. par contre :

```
int tab[23]={1, 12}; // le reste des cases est initialisé à 0.
```

Lecture, écriture

```
t[2] = ...; // écriture dans la 3ième cellule
... = t[9]; // lecture de la 10ième cellule
```

Passage de paramètre tableau

```
imprime_tableau(t); // et pas t[N] !
```

⚠ Toute modification du contenu tableau est enregistrée.

Tableaux de caractères Pas de "string" en C, mais des tableaux de char avec une sentinelle :

```
// déclaration et initialisation
char s[21]="hello"; // ajoute '\0' à la fin
```

On peut utiliser les fonctions de la bibliothèque string, ou parcourir comme ceci :

```
while(s[i] != '\0') { ...
```

8 Entrées-sorties

Sortie : impression sur le terminal (printf a un nombre de paramètres variable) :

```
printf("%d", x); // imprime la valeur entière contenue dans x
printf("Ou_alors_%c,%s\n", caractere, chaine); // autant de % que de variables à imprimer
```

Le premier argument est appelé *chaîne de formatage*.

Entrée : récupération d'une information entrée au clavier :

```
scanf("%d",&x); // modifie la valeur de x (int) avec la valeur entrée au clavier
scanf("%c",&c);
scanf("%s", s); // une chaîne est un tableau, pas de &
```

1 Compilation séparée

Prenons la structure classique suivante :

fonction.c fonction.h main.c

- fonction.h est un fichier d'en-tête qui contient des déclarations de fonction.
- fonction.c contient au moins l'implémentation de ces fonctions.
- Dans main.c, on inclut le fichier d'en-tête :

```
#include "fonction.h"
```

La compilation peut ensuite se faire de la façon suivante :

```
clang-14 -c fonction.c
clang-14 -c main.c
clang-14 -o nominaire main.o fonction.o
```

2 Nouveaux types de données

- **Types de base**: float, double (virgule flottante); signed char, unsigned char, unsigned int, ...
- **Les struct**: définition de type struct mys {int a; int b}; la définition d'une variable de ce type: struct mys vars;
- **Les enum**: définition du type enum week{Mond, Tues ...}; La définition d'une variable de ce type: enum week day;
- **Alias de type**: typedef, par exemple: typedef struct mys pair. La définition de variable ensuite: pair vars;

3 Le type pointeur

Le type int * est "adresse d'entier". Si t est un type, alors t* est le type "adresse vers un élément de type t".

Opérations :

- *p : contenu de la mémoire à l'adresse p.
- &x : adresse de la variable x.
- ==, != : comparaison

```
int x, y; //x,y sont des variables entières, elles sont des adresses.
y = 12; //initialisation de variable entière
int* pi; // définition du pointeur pi.
pi = &x; // initialisation de pi comme adresse (de x)
*pi = 2; // modification du contenu "pointé": x est modifié
pi = &y; // pi vaut maintenant l'adresse de y
*pi = *pi+1; // modif de contenu (de y)
pi = NULL; // valeur spéciale "vide"
```

Dans une expression, tout **tableau unidimensionnel** est remplacé par un **pointeur vers son premier élément** :

```
tab ≈ &tab[0]
tab+i ≈ &tab[i]
*tab ≈ tab[0]
```

4 Passage de paramètres

- Par **valeur** :

```
int f(int y);
```

L'appel f(23) effectue une copie de la valeur 23 dans le paramètre formel y. Les modifications sur y ne sont pas enregistrées.

- Par **adresse** : le contenu de l'adresse est modifiable.

Une fonction :

```
int g(int* y){
    *y=21;
    return 0;
}
```

Appel :

```
int u; //variable entière
g(&u); // &u: adresse d'entier
```

5 Allocation dynamique

```
#include <stdlib.h>
void* malloc (size_t size);
void free (void* ptr);
```

Effet :

- malloc alloue sur le tas un bloc de size octets, renvoie un pointeur **non typé (void*)** vers la zone allouée.
- Forcer le type de l'adresse avec un cast.
- free libère le bloc.
- le bloc est accessible uniquement par pointeur.

6 Tableaux

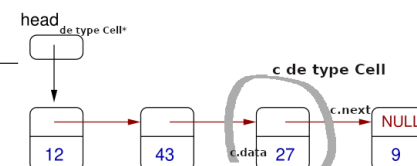
On dispose toujours de nos braves tableaux statiques, mais si jamais :

```
char *tab = NULL;

tab = malloc(42 * sizeof(char)); //42 octets demandés.
if (!arr) {
    perror("malloc");
    exit(EXIT_FAILURE);
}
tab[2] = ... //Écriture et lecture classiques.
```

7 Les listes chaînées

```
typedef struct Cell {
    Cell* next;
    int data;
} Cell ;
```



- data est le contenu de la cellule (ici un entier).
- next pointe vers la cellule suivante ou vaut NULL (fin de liste).
- On accède à la liste à l'aide d'un pointeur vers le premier élément (head sur le dessin).

Quelques éléments :

```
Cell* head = (Cell*) malloc(sizeof(Cell)); //définir et allouer une cellule
head -> data = 42; // mise à jour du contenu
(*head).data = 42; // syntaxe équivalente
head -> next = NULL; // --> liste de taille 1.
```

8 Entrées-sorties

Les fichiers :

```
FILE* fp = fopen("myfile.txt", "r");
char line[100];
fgets(line, 100, fp); //Lire une ligne
if(!feof) {
    //utiliser la ligne
}
fclose(fp); //à ne pas oublier
```

9 Paramètres de la ligne de commande

```
int main(int argc, char** argv){
    // argc = nombre d'arguments (le binaire compte 1)
    // chaque argv[i] (pour i de 1 à argc-1) est une chaîne
    for (int i=1; i< argc; i++){
        printf("l'argument_numero_%d_est_%s\n", i, argv[i]);
    }
    ...// penser à utiliser la fonction atoi
}
```

10 Opérations "bitstring"

```
unsigned int u=42;
u = u << 1; //shift par 1 (donc *2)
printf("value:%d(dec)\n", u); //84

// Faire un masque: 1111 ..... 1011 (bit 2 à 0)
unsigned int mask = ~(1u << 2); //1u = constante 1 unsigned
```

```
u = 15; // 0----0111
// Appliquer le masque
u = u & mask ; // positionne le bit 2 à 0. ("et" bit à bit)
printf("value:%d(dec)\n", u); //11
```